

6 Arrays and strings

In fact, you can even take a shortcut to the same compare:

```
String b = "HELLO";
"hello".equalsIgnoreCase( b );
```

```
String b = "hacker heaven";
String n = " ";
n = b.replace( 'h', 'H' );
```

Remember, this does not change the original, it return a new String object in which the changes have been made.

```
String b = " hacker heaven ";
String n = "";
n = b.trim();
{ n == "hacker heaven" }
```

This method merely removes any extra spaces from both the front and back of a String. Note that this does not alter the original String. Rather, it returns a new String with the leading and trailing spaces omitted.

6.4.5 StringBuffer Class

An object of class String cannot be altered after it has been created. An object of class StringBuffer can be altered. We describe the class StringBuffer as a mutable class, meaning it can be changed. StringBuffer are used internally to implement many of the methods in the String class. Every StringBuffer object can hold only a certain amount of characters. A StringBuffer has a capacity that describes the number of characters it can hold. This number is determined at the time of creation. If not specified when it is instantiated, a StringBuffer has a default capacity of 16 characters. Therefore, you generally specify the capacity when you instantiate your StringBuffer. We say a StringBuffer is Dynamically Resizable. A StringBuffer is instantiated as follows:

```
StringBuffer d = new StringBuffer();
This one will begin with a capacity of 16 characters.
StringBuffer s = new StringBuffer( 100 );
This one will begin with a capacity of 100 characters.
StringBuffer e;
e = new StringBuffer( "hello" );
```

This one has a capacity of 21 characters, because the initializing String contains 5 characters.